

MODULE-6:

Here's an explanation of the important points from the document "AWS-Module-6.pdf", simplified for better understanding:

AWS Compute Services Overview

This document explains AWS compute services. It covers what compute services AWS offers, how to choose the best compute service for your needs, how to optimize costs, and how to use specific compute services like Amazon EC2, AWS Lambda, and AWS Elastic Beanstalk.

AWS Compute Services

AWS has many compute services to meet various needs. Think of them in four categories:

- **Virtual Machines (VMs):** Like renting a whole computer in the cloud. You pick the operating system, size, and resources. You manage it like your own server. Example: Amazon EC2.
- **Serverless:** You just provide your code; AWS runs it when needed. No server management at all. Pay only for the time your code runs. Example: AWS Lambda.
- **Container-Based:** Apps run in isolated units called containers, sharing the same OS. Starts faster than VMs, good for responsiveness. Examples: Amazon ECS, Amazon EKS, AWS Fargate, Amazon ECR.
- **Platform as a Service (PaaS):** Focus just on your app's code. AWS handles everything else needed to run it (servers, databases, etc.). Example: AWS Elastic Beanstalk.

Choosing the Right Service

The best compute service depends on what you're building.

- **Consider:** Your app's design, how you'll use it, and what settings you want to control.
- **Legacy code:** Older apps might limit your choices, but you can evolve towards cloud-friendly designs.

Amazon EC2: Virtual Machines in the Cloud

- **Cost-Effective:** Cheaper than running physical servers.
- **Resizable:** Easily adjust the power of your virtual server.
- **Flexible:** Host a wide range of applications.
- **Control:** Full control over the operating system.
- **Security:** Control network access with security groups.

Launching an EC2 Instance

These are the key decisions you need to make:

1. **Amazon Machine Image (AMI):** The template for your virtual server. Choose the operating system and pre-installed software you need.
2. **Instance Type:** Determines the server's resources (CPU, RAM, storage, network). Choose based on your app's requirements.
3. **Network Settings:** Choose the virtual network and location (Region, Availability Zone) for your server. Decide if it needs a public IP address to be accessible from the internet.
4. **IAM Role (Optional):** Grant permissions to your server to access other AWS services.
5. **User Data Script (Optional):** Run custom commands when the server starts to configure it automatically.
6. **Storage Options:** Choose the type and size of storage for your server (e.g., Amazon EBS for permanent storage, Instance Store for temporary storage).
7. **Tags:** Add labels to your server for easy organization and management.
8. **Security Group Settings:** Configure firewall rules to control network traffic to your server.
9. **Key Pair:** Used to securely connect to your server.

Amazon EC2 Cost Optimization

- **Pricing Models:** AWS offers various ways to pay for EC2:
 - **On-Demand Instances:** Pay by the hour, no commitment.
 - **Spot Instances:** Bid on unused instances for the lowest price, can be interrupted.
 - **Reserved Instances:** Reserve instances for a longer term for a lower hourly rate.
 - **Dedicated Hosts:** Rent a whole physical server.
 - **Dedicated Instances:** Instances isolated from others at the hardware level.
- **Cost Optimization Pillars:**
 - **Right-Size:** Choose the right instance type and size to avoid paying for unused resources.
 - **Increase Elasticity:** Stop or hibernate instances when not needed, use automatic scaling to adjust capacity based on demand.
 - **Optimal Pricing Model:** Select the best pricing model for your usage patterns.
 - **Optimize Storage Choices:** Choose cost-effective storage options and delete unnecessary snapshots.

Container Services

- **Containers:** Pack applications with everything they need to run, making them portable and consistent across environments.
- **Docker:** Popular software platform for building and running containers.
- **Amazon ECS (Elastic Container Service):** Managed service to easily run Docker containers on AWS. You can choose to manage the infrastructure (EC2 launch type) or let AWS handle it (Fargate launch type).
- **Kubernetes:** Open-source software for managing containers at a larger scale, can be used on-premises or in the cloud.
- **Amazon EKS (Elastic Kubernetes Service):** Managed service to run Kubernetes on AWS.
- **Amazon ECR (Elastic Container Registry):** Store and manage Docker images for use with your container services.

AWS Lambda: Serverless Computing

- **Run Code Without Servers:** Just upload your code, AWS handles the infrastructure and runs it when needed.
- **Pay-per-use:** Only charged for the time your code runs.
- **Event-Driven:** Lambda functions are triggered by events, such as file uploads to S3 or changes in a database.
- **Schedule-Based:** You can also set your code to run on a schedule.

Lambda Use Cases

- **Process data in real-time:** Image resizing, data analysis, log processing.
- **Backend for web and mobile apps:** User authentication, data storage, API requests.
- **Automate tasks:** Start and stop EC2 instances, backup data, send notifications.

AWS Elastic Beanstalk: Platform as a Service (PaaS)

- **Easy Web App Deployment:** Upload your code, Elastic Beanstalk handles infrastructure setup, deployment, scaling, and monitoring.
- **Supports Various Platforms:** Deploy apps written in Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker.
- **Cost-Effective:** No charge for Elastic Beanstalk itself, pay only for the underlying AWS resources you use.

The document also includes a hands-on lab to guide you through setting up an EC2 instance and activities to practice using AWS Lambda and AWS Elastic Beanstalk.